RFID personnel tracking system

Workflow document — architecture, payloads, and data flow
Two deployment scenarios: fixed gate reader, and vehicle-mounted safety reader

1. System overview

This system covers two distinct deployments built on the same reader technology (Zebra FXR90, ZIOTC over MQTT) and the same badge registry, but with different physical topology, connectivity, and data semantics:

- **Scenario 1 — gate reader.** Fixed installation at the site gate. Reads badges as people pass, registers new ones, logs check-in events. Fully designed and tested (Sections 2, 4.1–4.5, 6 tags/checkins).
- **Scenario 2 — vehicle-mounted reader.** Reader and edge compute both ride on a moving vehicle, confirming registered, checked-in personnel are present on site as it patrols. Architecture defined; several open engineering questions remain before build (Section 3).

Both scenarios share the reader-side protocol (ZIOTC/MQTT, Section 4) and can share the same badge registry (Section 6), since the same population of personnel badges is read by both.

2. Scenario 1 — gate reader (fixed installation)

A Zebra FXR90 UHF RFID reader, fixed-mounted at the site gate, reads passive badges as people pass within its read range (rated up to roughly 30.5 m). The reader's built-in Zebra IoT Connector (ZIOTC) publishes tag reads as JSON over MQTT to a broker running locally on a Jetson Nano. The Jetson subscribes, deduplicates repeat reads, registers any badge it has never seen before, and logs a check-in event (debounced) for every pass — new or returning. Both events are sent to the cloud over HTTPS, where they land in a registry database.

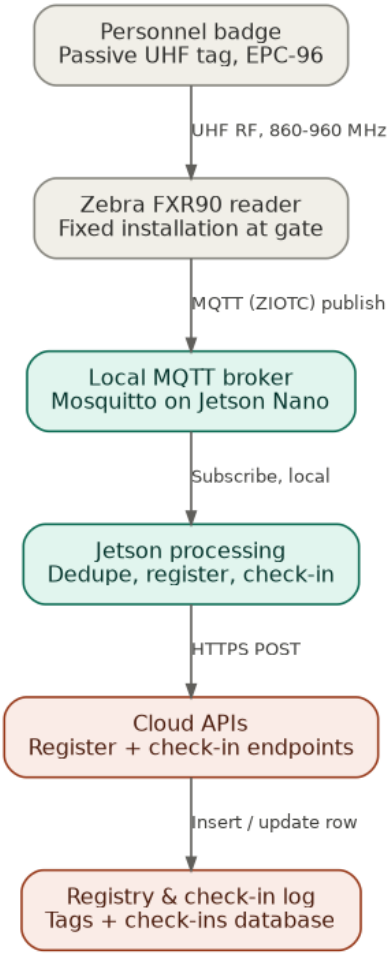

Figure 1. Complete data path from badge to cloud registry (Scenario 1).

2.1 Physical layer

Property	Value
Reader	Zebra FXR90, ultra-rugged fixed UHF reader
Mounting	Fixed installation at the gate (wall, frame, or stand — not pole-mounted)
Antenna ports	4 external
RF band	UHF, 860–960 MHz (worldwide)
Rated max read range	~30.5 m
Connectivity on reader	Gigabit Ethernet (primary), Wi-Fi 6, Bluetooth 5.3 (local config only)

2.2 Network & protocol architecture

Hop	Protocol	Notes
Reader -> Jetson	MQTT (ZIOTC Data + Management Events interfaces)	Broker runs locally on the Jetson (mosquitto). Reader is the publisher; Jetson is the subscriber.
Jetson -> Cloud	HTTPS POST	Simpler, no cloud broker to run. MQTT+TLS remains the stronger option if the site's internet is unstable (broker-side offline queuing, QoS acks).
Jetson control -> Reader	MQTT (ZIOTC Management/Control interface)	Command/response pattern, correlated by command_id. Not yet implemented.

3. Scenario 2 — vehicle-mounted reader (presence verification)

A second FXR90 reader, mounted on a pole on a vehicle that moves around the factory floor, reads personnel badges as it passes them — not a checkpoint, but a rolling presence sweep that confirms registered, checked-in personnel are still on site. This is not an alert-generation or collision-safety system; it is a logging system, structurally close to the gate's check-in flow.

This scenario is architecturally defined but not yet built or tested. Treat this section as a design specification, not a confirmed implementation like Section 2.

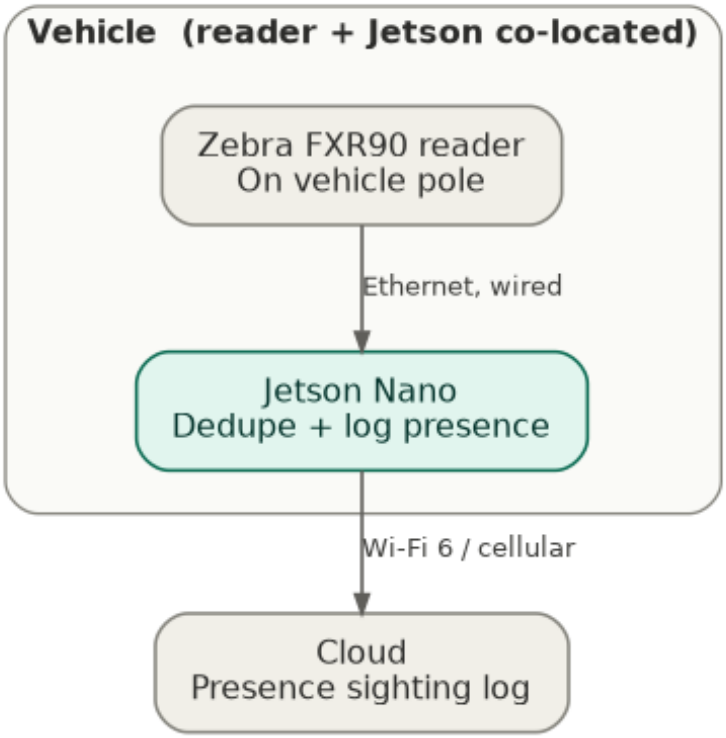

Figure 2. Vehicle-mounted architecture: reader and Jetson ride the vehicle, presence sightings are logged to the cloud (Scenario 2).

3.1 How this differs from Scenario 1

Physically, the reader and Jetson are co-located on the vehicle rather than fixed at a gate, and the Jetson's uplink to the cloud must be wireless (Wi-Fi or cellular) since the vehicle moves, rather than wired Ethernet. Functionally, however, the Jetson's job is nearly identical to the gate flow: subscribe to tag events, deduplicate repeat reads, and log a presence event to the cloud. There is no local decision-making or real-time response requirement on the vehicle — a sighting can be logged asynchronously, the same way a check-in is. In practice, the vehicle-side agent can reuse jetson_registrar.py's subscribe/dedupe/POST pattern directly, pointed at a different topic and a presence-sighting endpoint instead of a check-in endpoint.

3.2 Open questions before this can be built

- **No location data.** The vehicle has no positioning hardware, so a presence sighting only tells you a badge was seen by a specific vehicle, not where in the factory. presence_sightings does not carry a location field for this reason. If knowing where someone was last sighted becomes necessary later, that requires adding positioning hardware to the vehicle first — GPS outdoors, or an indoor approach (Wi-Fi RTT, BLE beacons, odometry) indoors, since factory buildings typically kill GPS accuracy — this is not something to infer from the RFID data itself.
- **Connectivity while moving.** The reader-to-Jetson link stays wired (both ride the vehicle), but the Jetson-to-cloud link must be wireless and handle roaming across the factory's Wi-Fi access points, or fall back to cellular where coverage is unreliable.
- **Power.** Both the reader and the Jetson need to run off the vehicle's electrical system — a properly sized 12V/24V DC-DC converter, not a USB power bank.
- **What counts as "still present."** If a checked-in badge hasn't been sighted by any reader (gate or vehicle) for some period, is that worth flagging as an anomaly? This is a reporting/dashboard decision, not something the vehicle itself needs to compute, but it determines what "presence verification" actually delivers to an operator.
- **Debounce window for a roaming reader.** The gate's 300 s debounce exists to avoid re-logging one walk-through many times. A vehicle patrolling the same area repeatedly may need a different window — short enough to reflect real presence, long enough not to flood the log.

4. Payload reference

The following payload shapes are shared by both scenarios — both readers are FXR90 units speaking the same ZIOTC protocol.

4.1 Tag data event — reader -> Jetson (confirmed schema)

Published on the reader's Data Interface topic every time a tag is read (mode-dependent; SIMPLE mode reports every unique tag seen).


```
{
  "type": "SIMPLE",
  "timestamp": "2026-07-10T09:14:02.331Z",
  "data": {
    "idHex": "E28011700000020F7C0A1B01",
    "format": "epc",
    "antenna": 1,
    "peakRssi": -39,
    "channel": 911.75,
    "phase": 0,
    "reads": 1,
    "eventNum": 1,
    "CRC": "b06a",
    "PC": "3000",
    "XPC": "string",
    "TID": "e20060030178529d",
    "USER": "1234...",
    "accessResults": ["SUCCESS"]
  }
}
```


Field	Meaning
type	Operating mode that produced this event (SIMPLE, INVENTORY, PORTAL, CONVEYOR, CUSTOM)
data.idHex	The tag's ID as hex — the EPC. This is the field registration/check-in is keyed on.
data.format	Which memory bank/format idHex represents
data.antenna	Antenna port that detected the tag (1–4) — zone/direction signal
data.peakRssi	Signal strength, dBm (closer to 0 = stronger)
data.channel	RF channel frequency, MHz — diagnostic only
data.phase	RF phase angle — used for angle-of-arrival positioning, not used here
data.reads	Times seen within the reporting interval
data.CRC / PC / XPC	Raw Gen2 protocol header fields — not used by application logic
data.TID	Chip's factory-burned serial (immutable, unlike the EPC)

Field	Meaning
data.USER	Contents of the tag's User memory bank, if configured to read it
data.accessResults	Result of any extra memory-bank read/write operations

4.2 Management event — heartbeat — reader -> Jetson (confirmed envelope)

Published periodically (default 60 s) on the reader's Management Events interface — health telemetry, independent of tag data.


```
{
  "type": "heartbeat",
  "timestamp": "2026-07-10T09:15:00Z",
  "component": "RG",
  "eventNum": 100,
  "data": {
    "radio_control": {
      "antennas": {"1": "connected", "2": "disconnected",
                   "3": "disconnected", "4": "disconnected"},
      "radioConnection": "connected",
      "radioActivity": "active",
      "numTagReads": 39,
      "status": "running",
      "uptime": "00:16:01"
    },
    "reader_gateway": {
      "interfaceConnectionStatus": {
        "data": [{"interface": "mqtt", "connectionStatus": "connected",
                  "connectionError": ""}]
      },
      "numErrors": 4
    },
    "system": {},
    "userapps": []
  }
}
```


radio_control and **reader_gateway** field examples above are confirmed from Zebra's docs. **system** typically carries CPU/memory/flash/temperature/NTP/uptime/power fields at the category level — exact keys unverified, confirm against a live reader. **userapps** lists any custom on-device apps (empty if none deployed).

Why this matters for Scenario 2: a vehicle-mounted reader that silently loses its radio or MQTT connection stops confirming anyone's presence with no indication anywhere that it's happened — a gap in the presence log could easily be mistaken for the reader working correctly and simply not encountering anyone. Heartbeat monitoring is worth prioritizing for the vehicle deployment even before the gate deployment.

4.3 Command / response schema — Jetson -> reader control (pattern, partially confirmed)

For remote start/stop or config changes over MQTT (Management/Control interface). Not yet implemented in jetson_registrar.py.

```
{
  "command": "start",
  "command_id": "16266718797272556",
  "payload": {}
}
```

command_id is generated by the sender and echoed back in the response, since MQTT has no native request/response pairing — this is how a reply gets matched to the request that triggered it. The response envelope's exact success/error field names are unverified — confirm at zebradevs.github.io/rfid-ziotc-docs/schemas/raw_mqtt_payloads/responses/ before building a strict parser against it.

4.4 Registration payload — Jetson -> cloud (Scenario 1, tested working)

Sent once per EPC, the first time it's ever seen, from either reader.

```
{
  "epc": "E28011700000020F7C0A1B01",
  "reader_id": "ZEBRA-FXR90-01",
  "site_id": "SITE-A",
  "first_seen_utc": "2026-07-10T17:44:59.315726+00:00"
}
```

POSTs to CLOUD_REGISTRATION_URL (/api/tags/register). Upserts into the tags table with status = 'unassigned'.

4.5 Check-in payload — Jetson -> cloud (Scenario 1, tested working)

Sent every time a badge is read at the gate, new or known, subject to the debounce window (default 300 s per EPC).

```
{
  "epc": "E28011700000020F7C0A1B01",
  "reader_id": "ZEBRA-FXR90-01",
  "site_id": "SITE-A",
  "antenna": 1,
  "rssi": -39,
  "checkin_ts": "2026-07-10T18:24:13.639224+00:00"
}
```

POSTs to CLOUD_CHECKIN_URL (/api/tags/checkin). Always inserts a new row into the checkins table (an append-only log, not an upsert).

4.6 Presence sighting payload — Jetson -> cloud (Scenario 2, proposed)

Proposed shape, not yet implemented or tested — matches the presence_sightings table in Section 6.

```
{
  "epc": "E28011700000020F7C0A1B01",
  "vehicle_id": "VEHICLE-03",
  "rssi": -28,
  "sighted_ts": "2026-07-10T18:24:13.639224+00:00"
}
```

No location field — the vehicle has no positioning hardware (Section 3.2), so this only records that VEHICLE-03 saw this badge, not where. Would POST to a new cloud endpoint, e.g. /api/presence/sighting — structurally the same idea as /api/tags/checkin (Section 4.5), just logged by a moving reader instead of a fixed one.

5. How we get each payload — step by step

Scenario 1 — tag data event

- Reader detects a badge in its RF field, reads the EPC.
- ZIOTC formats it per Section 4.1 and publishes to the configured Data Interface MQTT topic.
- Jetson's jetson_registrar.py is subscribed to that topic and receives it in on_message().

Scenario 1 — registration

- extract_tag_fields() pulls data.idHex out of the tag event.
- is_known(epc) checks the Jetson's local SQLite cache (known_tags).
- If not known: register_with_cloud() POSTs Section 4.4's payload, then remember_tag() writes it to the local cache so it's never registered twice.

Scenario 1 — check-in

- Regardless of whether the tag was new, seconds_since_last_checkin() checks the debounce window against the local cache.
- If outside the window (or never checked in before): checkin_with_cloud() POSTs Section 4.5's payload, then record_checkin() updates the local last-checkin timestamp.

Scenario 1 — heartbeat (proposed, not yet built)

- Jetson would subscribe to the reader's Management Events topic separately from the tag data topic.
- On each heartbeat, check reader_gateway.interfaceConnectionStatus and radio_control.radioConnection / antennas.
- If a heartbeat is late or reports a disconnected state, raise an alert.

Scenario 2 — presence verification (proposed, not yet built)

- Reader on the vehicle detects a badge, publishes a tag data event exactly as in Section 4.1, over the wired link to the on-vehicle Jetson.
- Jetson extracts data.idHex the same way as jetson_registrar.py does today (extract_tag_fields()), and checks it against a local debounce cache keyed by EPC.
- If outside the debounce window: Jetson posts Section 4.6's presence sighting payload to the cloud over whatever wireless link is available. This is ordinary asynchronous logging, with no on-vehicle decision or response required — the same shape as the gate's check-in flow (Section 5, Scenario 1), just sourced from a moving reader.
- If the link is down when a sighting occurs, the same outbox/retry pattern proposed for the gate (Section 8) applies here too — queue locally, retry once connectivity returns.

6. Cloud data model

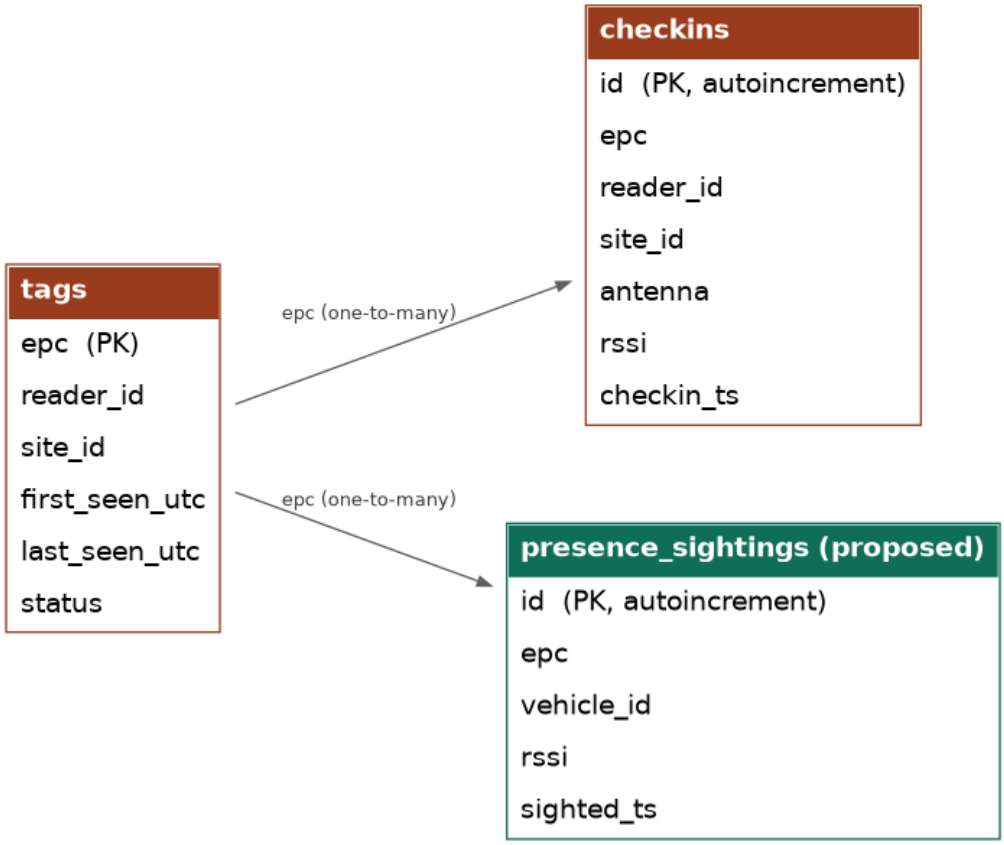

Figure 3. tags and checkins are built and tested (Scenario 1); presence_sightings is proposed (Scenario 2).

tags — the registry, one row per EPC ever seen, shared by both scenarios

Column	Type	Notes
epc	TEXT, PK	
reader_id	TEXT	
site_id	TEXT	
first_seen_utc	TEXT	Set once, on registration
last_seen_utc	TEXT	Updated on every registration payload
status	TEXT	'unassigned' until a person is linked to the badge

checkins — append-only arrival log (Scenario 1)

Column	Type	Notes
id	INTEGER, PK, autoincrement	
epc	TEXT	
reader_id	TEXT	
site_id	TEXT	
antenna	INTEGER	Which antenna port triggered this check-in
rssi	INTEGER	Signal strength at check-in
checkin_ts	TEXT	

presence_sightings — append-only presence log (Scenario 2, proposed)

Same shape as checkins, sourced from a moving reader instead of a fixed one. Together, the most recent row across both tables for a given EPC answers "when was this person last seen, and by which reader" — not where in the factory, since neither reader has positioning hardware.

Column	Type	Notes
id	INTEGER, PK, autoincrement	
epc	TEXT	
vehicle_id	TEXT	
rssi	INTEGER	
sighted_ts	TEXT	

7. Code reference

File	Role	Status
config.py	Broker address, reader topic, cloud URLs, debounce window, field-name constants	Built
jetson_registrar.py	Runs on the gate Jetson. MQTT subscriber, local dedupe cache, registration + check-in logic	Built, tested
cloud_registration_service.py	Stand-in cloud service. REST API + SQLite store for tags/checkins	Built, tested

File	Role	Status
(vehicle-side agent)	Would run on the vehicle Jetson: same subscribe/dedupe/POST pattern as jetson_registrar.py, reconfigured for the presence-sighting endpoint	Not started

8. Open decisions / next steps

Scenario 1 (gate)

- **MQTT vs HTTPS for the cloud leg** — currently HTTPS. MQTT is the stronger choice specifically for unstable-internet sites but requires running or paying for a cloud broker.
- **Outbox/retry pattern** — designed but not yet implemented. Both register_with_cloud() and checkin_with_cloud() currently fail silently on a dropped connection. Highest-priority gap before relying on this at a real gate.
- **Heartbeat/health monitoring** — not yet implemented. A dead radio or dropped MQTT client currently produces no alert.
- **EPC vs TID as the registration key** — currently keyed on idHex (EPC), which is technically rewritable. TID is immutable. Decide before registering real badges.
- **Check-in vs checkout** — currently entry-only.
- **Debounce window tuning** — defaults to 300 s, not yet validated against real foot traffic.

Scenario 2 (vehicle)

- **Positioning (not in scope now)** — presence_sightings deliberately has no location field, since the vehicle has no positioning hardware. If "where," not just "when," becomes necessary later, that means adding GPS, indoor RTT/beacons, or odometry to the vehicle first, and a schema change after.
- **Wireless roaming behavior** — needs validation that the Jetson's Wi-Fi connection survives moving between access points without dropping the MQTT session to the reader or stalling presence logging.
- **Vehicle power design** — DC-DC converter sizing for reader + Jetson off the vehicle's electrical system.
- **Cloud endpoint for presence sightings** — not yet built, mirrors the pattern in cloud_registration_service.py but as a new route.
- **Staleness reporting** — what it means, and who is told, when a checked-in badge hasn't been sighted by any reader in an unusually long time. A dashboard/reporting decision, not a vehicle-side computation.
- **Debounce window for a roaming reader** — likely different from the gate's 300 s default, not yet validated.